

ROS2

DAY 5 – Manual 1

Actions – Implementation (Fibonacci Example)

Step-by-Step Code, Explanation & Execution

Objective

By the end of this manual, you will be able to:

- Write a Python Action Server
 - Write a Python Action Client
 - Understand all important ROS2 Action APIs
 - Build the workspace
 - Run server & client
 - Observe feedback and result
 - Debug common issues
-

Project Structure (Final Workspace)

Inside `ros2_ws/src/`:

```
ros2_ws/  
└─ src/  
    └─ action_pkg    # Interface package (Fibonacci.action)  
    └─ my_py_pkg     # Python package (server + client)
```

Step 1 – Create Python Package for Action Nodes

```
cd ~/ros2_ws/src  
ros2 pkg create my_py_pkg --build-type ament_python
```

This auto-creates:

```
my_py_pkg/  
├── my_py_pkg/  
│   ├── __init__.py  
│   ├── action_server.py  
│   └── action_client.py  
...
```

Step 2 – Update setup.py

Open:

```
nano my_py_pkg/setup.py
```

Add your scripts:

```
entry_points={  
    'console_scripts': [  
        'fibonacci_server = my_py_pkg.action_server:main',  
        'fibonacci_client = my_py_pkg.action_client:main',  
    ],  
},
```

Why?

So ROS2 can run:

```
ros2 run my_py_pkg fibonacci_server
```

Step 3 – Implement the Action Server

Create the file:

```
nano my_py_pkg/my_py_pkg/action_server.py
```

Paste:

```
import rclpy  
from rclpy.node import Node  
from rclpy.action import ActionServer  
from rclpy.executors import MultiThreadedExecutor
```

```
from action_pkg.action import Fibonacci
```

```
class FibonacciActionServer(Node):
```

```
    def __init__(self):  
        super().__init__('fibonacci_action_server')
```

```
  
        self._action_server = ActionServer(  
            self,  
            Fibonacci,  
            'fibonacci',  
            self.execute_callback  
        )
```

```
  
        self.get_logger().info("Fibonacci Action Server is ready!")
```

```
    def execute_callback(self, goal_handle):
```

```
        self.get_logger().info(f'Received goal: order = {goal_handle.request.order}')
```

```
        # Prepare result & feedback
```

```
        feedback_msg = Fibonacci.Feedback()
```

```
        result = Fibonacci.Result()
```

```
        sequence = [0, 1]
```

```
        feedback_msg.partial = sequence.copy()
```

```
        # Publish initial feedback
```

```
        goal_handle.publish_feedback(feedback_msg)
```

```
        self.get_logger().info(f'Feedback: {feedback_msg.partial}')
```

```
        # Generate Fibonacci sequence
```

```
        for i in range(2, goal_handle.request.order):
```

```
            sequence.append(sequence[i - 1] + sequence[i - 2])
```

```
            feedback_msg.partial = sequence.copy()
```

```
            goal_handle.publish_feedback(feedback_msg)
```

```
            self.get_logger().info(f'Feedback: {feedback_msg.partial}')
```

```
        result.sequence = sequence
```

```
        goal_handle.succeed()
```

```
        self.get_logger().info(f'Sending result: {result.sequence}')
```

```
        return result
```

```
def main(args=None):
```

```
    rclpy.init(args=args)
```

```
    node = FibonacciActionServer()
```

```
executor = MultiThreadedExecutor()
executor.add_node(node)
```

```
try:
    executor.spin()
except KeyboardInterrupt:
    pass
```

```
node.destroy_node()
rclpy.shutdown()
```

Step 4 – Implement the Action Client

Create:

```
nano my_py_pkg/my_py_pkg/action_client.py
```

Paste:

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient

from action_pkg.action import Fibonacci

class FibonacciActionClient(Node):
    def __init__(self):
        super().__init__('fibonacci_action_client')
        self._client = ActionClient(self, Fibonacci, 'fibonacci')

    def send_goal(self, order):
        self._client.wait_for_server()

        goal_msg = Fibonacci.Goal()
        goal_msg.order = order

        self.get_logger().info(f"Sending goal: order={order}")

        return self._client.send_goal_async(
            goal_msg,
            feedback_callback=self.feedback_callback
        )

    def feedback_callback(self, feedback_msg):
```

```

        self.get_logger().info(f"Feedback: {feedback_msg.feedback.partial}")

    def get_result(self, goal_future):
        goal_handle = goal_future.result()

        if not goal_handle.accepted:
            self.get_logger().error("Goal rejected!")
            return

        self.get_logger().info("Goal accepted.")

        result_future = goal_handle.get_result_async()
        rclpy.spin_until_future_complete(self, result_future)

        return result_future.result().result

def main(args=None):
    rclpy.init(args=args)

    node = FibonacciActionClient()

    goal_future = node.send_goal(10)
    rclpy.spin_until_future_complete(node, goal_future)

    result = node.get_result(goal_future)
    print(f"Final Result: {result.sequence}")

    node.destroy_node()
    rclpy.shutdown()

```

Step 5 – Install Dependencies in setup.py

Ensure:

```

install_requires=['setuptools'],
zip_safe=True,

data_files=[
    ('share/ament_index/resource_index/packages',
     ['resource/my_py_pkg']),
    ('share/my_py_pkg', ['package.xml']),
],
(ROS auto-generated parts stay unchanged.)

```

Step 6 – Build Sequence (VERY IMPORTANT)

Because actions are interfaces, build order matters.

1. Build action_pkg FIRST

```
cd ~/ros2_ws  
colcon build --packages-select action_pkg  
source install/setup.bash
```

2. Then build Python package

```
colcon build --packages-select my_py_pkg  
source install/setup.bash
```

Step 7 – Run the Nodes

Terminal 1: Start Action Server

```
ros2 run my_py_pkg fibonacci_server
```

Expected:

```
[FibonacciActionServer] Fibonacci Action Server is ready!  
Received goal: order=10  
Feedback: [0, 1]  
Feedback: [0, 1, 1]  
...
```

Terminal 2: Start Action Client

```
ros2 run my_py_pkg fibonacci_client
```

Expected:

```
[INFO]: Sending goal: order=10  
[INFO]: Feedback: [0, 1]  
[INFO]: Feedback: [0, 1, 1]  
...  
Final Result: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Step 8 – Debugging

✗ Error: "Unknown action type"

Solution:

```
source install/setup.bash
```

✗ ActionServer not receiving goal

Check package names:

- action_pkg/action/Fibonacci.action
- Python import:
- from action_pkg.action import Fibonacci

✗ Client stuck "waiting for server"

Check:

```
ros2 action list
```

You should see /fibonacci.

Step 9 – Quick Start for Repo Users

```
git clone https://github.com/myt-mthn/ROS2-Tutorials.git
cd ROS2-Tutorials/DAY-05/ros2_ws
colcon build
source install/setup.bash
```

🎉 Manual 2 Complete!